MINOR PROJECT REPORT

On

A Local Student Opportunity & Networking Platform

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Guided By:

[GUIDE_NAME]
Assistant Professor

Submitted By:

[STUDENT_NAME]
Roll No: [ROLL_NO]
Enrollment No: [ENROLLMENT_NO]

MCA 2nd Semester | 2025

Department of Computer Science and Information Technology

Guru Ghasidas Vishwavidyalaya, Bilaspur (C.G.)

This is to certify that the Minor Project Report entitled **"MyCircle – A Local Student Opportunity & Networking Platform"** submitted by **[STUDENT_NAME]**, Roll No: [ROLL_NO], MCA 2nd Semester student at Guru Ghasidas Vishwavidyalaya, Bilaspur (C.G.), is a bonafide record of the project work carried out by him/her under my supervision and guidance during the academic year 2025.

The project has been completed in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications and is of sufficient merit to warrant the candidate's appearance for the viva-voce examination.

The candidate has fulfilled all prescribed conditions and has demonstrated thorough understanding of modern full-stack web development principles, system design, and software engineering practices throughout the project.

Date: _____

Place: Bilaspur (C.G.)

(Signature of Candidate)

[STUDENT_NAME]

(Signature of Guide)

[GUIDE_NAME]

I, **[STUDENT_NAME]**, hereby declare that the Minor Project Report entitled **"MyCircle – A Local Student Opportunity & Networking Platform"** submitted by me is an authentic record of my project work carried out at the Department of Computer Science and Information Technology, Guru Ghasidas Vishwavidyalaya, Bilaspur (C.G.), under the guidance of **[GUIDE_NAME]**, Assistant Professor.

I further declare that this project work or any part thereof has not been submitted for the award of any degree, diploma, or certificate either to this University or to any other University or Institution.

I assure that the code, algorithms, implementation, and documentation presented in this report are original and created by me through proper understanding, learning, and implementation of the required concepts and technologies.

Date: _____

Place: Bilaspur (C.G.)

(Signature of Candidate)

[STUDENT_NAME]

(Signature of Guide)

[GUIDE_NAME]

I express my sincere gratitude and heartfelt thanks to the Head of Department, Department of Computer Science and Information Technology, Guru Ghasidas Vishwavidyalaya, Bilaspur (C.G.), for providing the necessary facilities, resources, and academic environment required to carry out this project successfully.

I am deeply grateful to my project guide, **[GUIDE_NAME]**, Assistant Professor, for his invaluable guidance, constant motivation, and constructive criticism throughout the development and documentation phases of this project. His technical insights and patient mentorship significantly elevated the quality of both the implementation and this report.

I also thank all the faculty members of the Department of Computer Science and Information Technology for their continuous support and for creating an inspiring learning environment throughout the MCA programme. Their collective wisdom has shaped my approach to problem-solving and software engineering.

My sincere thanks go to my classmates and friends who actively participated in testing MyCircle during its development phase, providing genuine feedback that helped identify usability issues and improve the platform's overall user experience.

I would like to acknowledge the open-source community and the dedicated documentation teams behind MongoDB, Express.js, React.js, Node.js, and all other technologies used in this project. These freely available resources were instrumental in every phase of development.

Finally, I express my profound gratitude to my parents and family for their unwavering support, encouragement, and belief in my abilities throughout my academic journey. Their sacrifices and motivation remain my greatest driving force.

The landscape of higher education in India has undergone significant transformation, with students increasingly requiring digital infrastructure that serves their hyper-local college ecosystems. While global platforms such as LinkedIn address professional networking at a macro level, and messaging applications like WhatsApp enable informal communication, no dedicated platform exists specifically for college students to discover local opportunities, connect with institutional peers, and engage meaningfully within their immediate academic community.

MyCircle is a comprehensive local student opportunity and networking platform developed using the MERN stack — MongoDB, Express.js, React.js, and Node.js. The platform enables students to post and discover local opportunities including internships, part-time jobs, freelance gigs, campus events, hackathons, and study groups. Students build verified profiles linked to their institution, follow their peers to create personalised feeds, communicate through real-time direct messaging, and stay informed through an intelligent notification system.

The frontend is built with React.js and Vite, styled using Tailwind CSS for a modern mobile-first responsive design. The backend employs a RESTful API architecture with Node.js and Express.js, managing data persistence through MongoDB with Mongoose ODM. Security is enforced through JWT-based authentication, bcrypt password hashing, and role-based access control. Real-time features are implemented using Socket.io, while Cloudinary handles cloud-based media storage.

This report documents the complete development lifecycle of MyCircle, covering requirements analysis, system design with ER diagrams and DFDs, technology stack analysis, feature implementation walkthrough, comprehensive testing with 20 test cases, and a realistic assessment of current limitations alongside a roadmap for future enhancement. The platform represents a functional prototype demonstrating the viability of hyper-local student networking tailored specifically to tier-2 and tier-3 city college ecosystems in India.

Keywords: **MERN Stack, Student Networking, Local Opportunities, React.js, MongoDB, Express.js, Node.js, JWT Auth**

Front Matter

Certificate of Approval	ii
Self Declaration	iii
Acknowledgements	iv
Abstract	v
Table of Contents	vi
List of Figures	vii
List of Abbreviations	vii i

Chapter 1: Introduction

1.1 Introduction to the Project	1
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Project Scope	4
1.5 Organization of the Report	5

Chapter 2: Literature Review / Existing Systems

2.1 Overview of Existing Platforms	6
2.2 Comparative Analysis	8
2.3 Research Gap	9
2.4 Justification for Building MyCircle	1 0

Chapter 3: System Requirements

3.1 Functional Requirements	1 1
3.2 Non-Functional Requirements	1 3
3.3 Hardware Requirements	1 4
3.4 Software Requirements	1 4

Chapter 4: System Design

4.1 System Architecture	1 5

4.2 Architecture Diagram 16
4.3 Database Design 17
4.4 ER Diagram 19
4.5 Data Flow Diagram (DFD) 20
4.6 Use Case Diagram 22
4.7 Module Description 23
4.8 API Design (REST Endpoints) 24
Chapter 5: Technology Stack
5.1 MongoDB & Mongoose 26
5.2 Express.js 27
5.3 React.js 28
5.4 Node.js 29
5.5 Vite 30
5.6 Tailwind CSS 31
5.7 JWT Authentication & bcrypt 31
5.8 Axios 32
5.9 Socket.io 33
5.10 Cloudinary & Git/GitHub 33
Chapter 6: Implementation & Feature Walkthrough
6.1 User Registration and Authentication 34
6.2 Student Profile Page 36

6.3 Opportunities Feed 37
6.4 Post an Opportunity 38
6.5 Campus Events 39
6.6 Study Groups 40
6.7 Follow / Unfollow System 41
6.8 Messaging 41
6.9 Notifications 42
6.10 Admin Panel 43
Chapter 7: Testing
7.1 Testing Strategy 44
7.2 Unit Testing 44
7.3 API Testing — Test Cases 45
7.4 Frontend Testing 47
7.5 Security Testing 47
7.6 Performance Testing 48
7.7 User Acceptance Testing 48
7.8 Bug Report 49
Chapter 8: UI Screenshots / Walkthrough
8.1–8.8 Screen Descriptions 50
Chapter 9: Limitations
9.1–9.4 Limitations 53

Chapter 10: Future Scope

5

10.1–10.9 Planned Enhancements 5

Chapter 11: Conclusion

5

11.1–11.4 Conclusion 8

References

6

Web and Technical References 0

Figure 4.1	System Three-Tier Architecture Diagram
Figure 4.2	User Authentication Flow Flowchart
Figure 4.3	DFD Level 0 — Context Diagram
Figure 4.4	DFD Level 1 — Process-Level Diagram
Figure 4.5	Entity-Relationship (ER) Diagram
Figure 8.1	Login and Registration Page Screenshot Placeholder
Figure 8.2	Home / Opportunity Feed Screenshot Placeholder
Figure 8.3	Student Profile Page Screenshot Placeholder
Figure 8.4	Post Opportunity Form Screenshot Placeholder
Figure 8.5	Campus Events Page Screenshot Placeholder
Figure 8.6	Study Groups Page Screenshot Placeholder
Figure 8.7	Direct Messaging Interface Screenshot Placeholder
Figure 8.8	Admin Dashboard Screenshot Placeholder

LIST OF TABLES

Table 2.1	Comparative Analysis of Existing Platforms
Table 3.1	Hardware Requirements
Table 3.2	Software Requirements and Tools
Table 4.1	Module Description Table
Table 4.2	REST API Endpoints (33 Endpoints)
Table 7.1	API Test Cases (TC-001 to TC-020)
Table 7.2	Bug Report Table

Abbreviation	Full Form
API	Application Programming Interface
bcrypt	Blowfish Crypt — Password Hashing Algorithm
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
ER	Entity-Relationship
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
MCA	Master of Computer Applications
MERN	MongoDB, Express.js, React.js, Node.js
MVC	Model-View-Controller
NoSQL	Not Only SQL
ODM	Object Document Mapper
ORM	Object Relational Mapper
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language

UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WS	WebSocket
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Introduction to the Project

The digital revolution has transformed how individuals connect, collaborate, and seek professional opportunities in unprecedented ways. While global platforms such as LinkedIn, Facebook, and Twitter have fundamentally reshaped professional and social networking at a macro level, a significant and underserved gap remains at the local level — specifically within the college and university ecosystems where students spend the most formative years of their professional development.

MyCircle emerges as a purpose-built solution designed exclusively for local college and university student communities. It is a comprehensive full-stack web platform that enables students to post and discover local opportunities — including internships, part-time jobs, freelance gigs, campus events, hackathons, workshops, and study groups — within their immediate institutional circle. The platform transcends the limitations of generic social media by offering specialised features designed for the unique needs and rhythms of student communities.

Students can create verified profiles linked to their institution, follow their peers to build personalised feeds, engage in real-time direct messaging, receive targeted notifications, and actively participate in their local student ecosystem. The platform is built on the MERN stack — MongoDB, Express.js, React.js, and Node.js — ensuring a modern, scalable, and maintainable codebase.

1.2 Problem Statement

Students in local colleges and universities face significant challenges in accessing relevant opportunities within their immediate geographic and institutional circles. The existing digital solutions either target an overly professional audience removed from student realities, or provide only casual social interaction without structured opportunity discovery capabilities.

Professional networking platforms like **LinkedIn**, while excellent for experienced professionals, are intimidating and often irrelevant for first and second-year students with limited professional history. **WhatsApp groups**, while popular among students, suffer from fundamental structural limitations — important information gets buried in linear threads, there is no effective search or categorisation, and opportunities vanish within hours of posting. **Internship platforms like Internshala** operate at a national level, completely missing local freelance gigs, study groups, events, and informal opportunities that form a substantial portion of the student landscape. Institutional portals are often outdated, lack social features, and are used primarily for

administrative communications.

The absence of a hyper-local, student-centric platform results in missed opportunities, fragmented communication, and an inability to build meaningful connections within one's immediate academic community — particularly acute in tier-2 and tier-3 city colleges where national platforms provide minimal local relevance.

1.3 Motivation

The motivation behind building MyCircle stems from direct observation of the challenges faced by fellow students in discovering local opportunities and connecting with peers within their college ecosystem. At institutions like Guru Ghasidas Vishwavidyalaya, students possess as much talent and ambition as their counterparts at premium institutions, but lack access to equivalent networks and information channels.

The local student ecosystem — comprising part-time work, campus events, peer collaborations, and study groups — is vibrant and active, but operates primarily through fragmented word-of-mouth and informal group chats. A centralised platform for this ecosystem could multiply the effectiveness of student efforts dramatically.

From an academic perspective, building MyCircle provides comprehensive exposure to the entire modern web development lifecycle — database design, API architecture, frontend development, authentication, cloud integration, and deployment. This project simultaneously solves a real-world problem and serves as a deep technical learning exercise preparing for professional software development roles.

1.4 Project Scope

The current version of MyCircle delivers a functional prototype demonstrating the core concept of local student networking. The scope is carefully defined to ensure quality delivery within minor project constraints while remaining genuinely useful.

- Complete JWT-based user authentication (registration, login, logout)
- Student profile creation with profile picture upload via Cloudinary
- Opportunity posting and discovery (internships, part-time, freelance, jobs)
- Campus event creation, listing, and management
- Study group creation, browsing, and membership management
- Follow/unfollow system with personalised activity feeds
- Real-time direct messaging between students
- Notification system for follows, posts, and event reminders
- Admin panel with user and content moderation capabilities

- Fully responsive design for desktop and mobile devices

Out of scope for this version: Mobile application (React Native), AI-based recommendation engine, Elasticsearch integration, email/OTP verification, two-factor authentication, payment gateway, and video calling.

1.5 Organization of the Report

This report is structured across eleven chapters covering the complete development lifecycle of MyCircle. Chapter 1 establishes context and motivation. Chapter 2 reviews existing platforms and identifies the research gap. Chapter 3 details functional and non-functional requirements. Chapter 4 presents system design including architecture, ER diagrams, and DFDs. Chapter 5 provides comprehensive analysis of the technology stack. Chapter 6 offers a complete feature implementation walkthrough. Chapter 7 documents the testing strategy and results. Chapter 8 provides UI screenshot documentation. Chapters 9 and 10 honestly assess limitations and future scope respectively. Chapter 11 concludes with key learnings and impact assessment.

Chapter 2

Literature Review / Existing Systems

2.1 Overview of Existing Platforms

LinkedIn: LinkedIn is the dominant global professional networking platform with over 900 million users. While excellent for connecting experienced professionals with corporate opportunities, LinkedIn caters primarily to career professionals with established work histories. Student profiles appear sparse and underdeveloped. The platform lacks features for local college communities, study groups, or the informal opportunities that define student experiences.

Internshala: Internshala is a leading Indian internship and fresher job platform offering structured opportunity listings with filtering and application tracking. However, it focuses exclusively on formal internships and jobs, completely overlooking local freelance work, campus events, study groups, and peer collaborations. It operates at national level without any local community features.

WhatsApp Groups: WhatsApp serves as the default student communication platform. While its instant messaging is highly effective, its architecture was not designed for opportunity discovery. Important posts disappear in message floods, there is no categorisation or search, no persistent listings, and no way to assess the credibility of shared opportunities.

College Portals: Institutional portals carry official credibility but suffer from severely outdated user experiences, minimal student-to-student interaction features, no opportunity posting mechanisms, and negligible mobile support. They exist primarily for administrative communications rather than community building.

Discord: Discord offers sophisticated community features including organised channels, voice rooms, and bot integrations. However, it is interest-based rather than institution-based, lacks verified student profiles, has no opportunity posting features, and its complexity can be overwhelming for casual users.

2.2 Comparative Analysis

The following table systematically compares MyCircle with existing platforms across ten critical dimensions that determine effectiveness for local student communities. Green indicates full support, orange indicates partial support, and red indicates absence.

Feature	LinkedIn	Internshala	WhatsApp	College Portal	Discord	MyCircle
Student-focused	Partial	Yes	Yes	Yes	No	Yes

Local College Focus	No	No	Partial	**Yes**	No	**Yes**
Opportunity Posting	Partial	**Yes**	No	No	No	**Yes**
Verified Profiles	Partial	No	No	**Yes**	No	**Yes**
Real-time Chat	No	No	**Yes**	No	**Yes**	**Yes**
Event Management	No	No	No	Partial	Partial	**Yes**
Study Groups	No	No	Partial	No	Partial	**Yes**
Notifications	**Yes**	Partial	**Yes**	No	**Yes**	**Yes**
Mobile Responsive	**Yes**	**Yes**	**Yes**	No	**Yes**	**Yes**
Free to Use	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Table 2.1 — Comparative Analysis of Existing Platforms

2.3 Research Gap

The comparative analysis reveals three specific research gaps. First, no existing platform provides a comprehensive local student opportunity directory that includes the complete spectrum — internships, part-time work, freelance gigs, campus events, hackathons, and study groups — within a single application. Second, no platform combines institutional profile verification with genuine social networking features tailored specifically to student communities. Third, existing platforms treat students either as job seekers (opportunity platforms) or casual social users (messaging apps), missing the collaborative community dimension where students both contribute to and benefit from each other's growth.

2.4 Justification for Building MyCircle

MyCircle is justified as a dedicated platform filling the identified research gap. Designed from the ground up for local college communities, it integrates opportunity discovery, event management, study groups, and peer networking into one cohesive experience. The MERN stack provides a modern, scalable foundation, and the mobile-first design ensures accessibility on devices most students carry. By focusing on local hyper-relevance rather than national breadth, MyCircle delivers value that no existing platform can match for its target audience.

Chapter 3 System Requirements

3.1 Functional Requirements

The following functional requirements define the complete set of features MyCircle must provide. These are derived from the problem statement, target user analysis, and platform scope defined in Chapter 1.

User Authentication

- Secure registration with name, email, password, college, and year of study
- JWT-based login with token issuance on successful credential verification
- Password hashing using bcrypt with minimum 10 salt rounds
- Persistent sessions across browser refreshes via localStorage
- Logout functionality clearing all tokens and session data

Profile Management

- Student profile creation with all academic and personal details
- Profile picture upload through Cloudinary with preview before save
- Skills/interests tag system for discoverability
- View follower count, following count, and post count on profiles
- Edit all profile information through an intuitive edit interface

Opportunities Module

- Create opportunity posts: internship, part-time, freelance, job categories
- Browse paginated opportunity feed with type, city, and search filters
- View detailed opportunity with application link and deadline
- Edit and delete own posts with owner verification middleware
- Bookmark and share opportunity links

Events Module

- Create campus events with title, date, venue, and registration details
- Chronological event listing with upcoming/past separation
- Event detail pages with organiser profile links
- Express interest functionality for event attendance tracking

Study Groups

- Create study groups with name, subject, description, and college scope
- Browse and search groups by subject and institution
- Join and leave groups with membership management
- View complete member list for each group

Social Features

- Follow and unfollow other students with real-time feed updates
- Personalised feed prioritising content from followed users
- View followers and following lists with profile links

Messaging & Notifications

- Direct messaging between any two authenticated users
- Conversation list with unread message indicators
- Real-time notifications for new followers, posts, and event reminders
- Notification bell with unread count badge and mark-as-read functionality

Admin Panel

- Dashboard with platform statistics: users, posts, events, reports
- User management: search, ban, and delete user accounts
- Content moderation: remove inappropriate or spam posts

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	Page load time under 3 seconds on standard broadband. API response time under 500ms. Support minimu
Security	Passwords hashed with bcrypt (10+ salt rounds). JWT tokens with configurable expiration. Protected routes
Scalability	Stateless REST API enabling horizontal scaling. MongoDB document model supporting sharding. Stateless
Availability	Target 99% uptime in production. Graceful error handling preventing complete system failures. Database co
Usability	Intuitive navigation without user training. Real-time form validation with clear error messages. Touch targets
Maintainability	Consistent naming conventions across codebase. Modular architecture enabling feature addition. Documen

3.3 Hardware Requirements

Requirement	Development Machine	Deployment Server
Processor	Intel Core i5 / AMD Ryzen 5 or higher	Dual-core CPU (min 1 GHz)

RAM	8 GB minimum (16 GB recommended)	512 MB minimum (1 GB recommended)
Storage	50 GB SSD free space	Free-tier cloud (Render / Railway)
Internet	Broadband (5 Mbps min)	Stable hosting connection
OS	Windows 10/11, Ubuntu 20.04, macOS 12+	Ubuntu 20.04 LTS (Server)
Display	1366 x 768 or higher	N/A (Headless server)

Table 3.1 — Hardware Requirements

3.4 Software Requirements

Software / Tool	Version	Purpose
Node.js	v18.x LTS	JavaScript server-side runtime environment
Express.js	v4.18.x	Minimalist web framework for REST API
React.js	v18.x	Frontend component-based UI library
MongoDB	v6.x	NoSQL document database (via Atlas)
Mongoose	v7.x	ODM for MongoDB schema and query building
Vite	v5.x	Fast frontend build tool and dev server
Tailwind CSS	v3.x	Utility-first CSS framework for styling
JWT (jsonwebtoken)	v9.x	Token-based authentication library
bcryptjs	v2.x	Password hashing with salt rounds
Axios	v1.x	Promise-based HTTP client for React
Socket.io	v4.x	Real-time bidirectional event communication
Cloudinary SDK	v1.x	Cloud image and media storage integration
Postman	v10+	REST API endpoint testing and documentation
VS Code	Latest	Primary Integrated Development Environment
Git / GitHub	v2.x	Version control and remote repository hosting
MongoDB Compass	Latest	GUI client for MongoDB database inspection

Table 3.2 — Software Requirements and Tools

Chapter 4

System Design

4.1 System Architecture

MyCircle employs a three-tier architecture that cleanly separates concerns and enables independent development and scaling of each layer. The three tiers are: the Client Tier (React.js Single Page Application), the API Tier (Node.js / Express.js REST Server), and the Data Tier (MongoDB Atlas + Cloudinary).

The **Client Tier** comprises the React.js SPA built with Vite. All user interactions, form submissions, and data display are handled here. React components communicate with the API tier exclusively through HTTP requests via Axios, with Socket.io maintaining a persistent WebSocket connection for real-time features.

The **API Tier** is the Node.js/Express.js server handling all business logic, authentication enforcement, and database orchestration. Every incoming request traverses a middleware chain: CORS validation, JSON body parsing, JWT authentication verification, route-specific handling, and error management. Controllers implement the MVC pattern with clean separation of routing, business logic, and data access.

The **Data Tier** consists of MongoDB Atlas for document storage managed through Mongoose ODM, and Cloudinary for all media assets. Mongoose provides schema validation, relationship references, indexing, and query building. Cloudinary provides scalable cloud image storage with transformation capabilities.

4.2 System Architecture Diagram

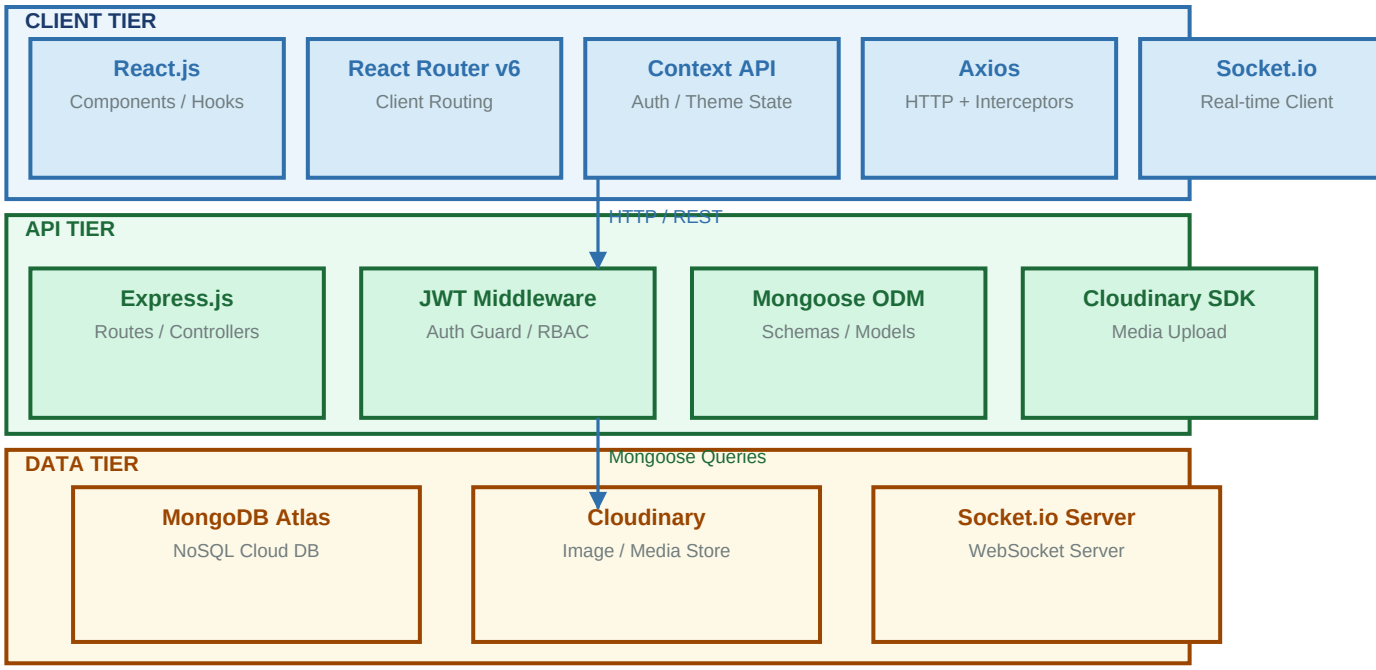

Figure 4.1 — Three-Tier System Architecture of MyCircle

4.3 Database Design

MyCircle uses MongoDB's flexible document model rather than a relational schema. Documents within each collection store JSON-like objects. This eliminates object-relational mapping complexity, aligns naturally with JavaScript's data structures, and allows schema evolution without expensive migrations. Six primary collections are defined:

Collection	Schema Fields
Users	_id, name, email (unique), password_hash, college, course, year, bio, profileImage (URL), role (student/a
Opportunities	_id, title, type (internship/part-time/freelance/job), description, postedBy (User ref), college, city, tags[ ], re
Events	_id, title, description, date, venue, organizer (User ref), college, registrationLink, isPublished, createdAt, u
StudyGroups	_id, name (unique), subject, description, admin (User ref), members[ ] (User refs), college, createdAt, upd
Messages	_id, sender (User ref), receiver (User ref), content, timestamp, read (boolean)
Notifications	_id, userId (User ref), type, message, relatedEntity (ref), isRead (boolean), createdAt

4.4 Entity-Relationship (ER) Diagram

The ER diagram illustrates the relationships between all six collections in the MyCircle database. Users are the central entity, maintaining 1:N relationships with Opportunities, Events, Notifications, and Study Groups (as admin). A M:N relationship exists between Users for the

follow system and between Users and Study Groups for membership. The Messages collection represents a M:N self-relationship on Users.

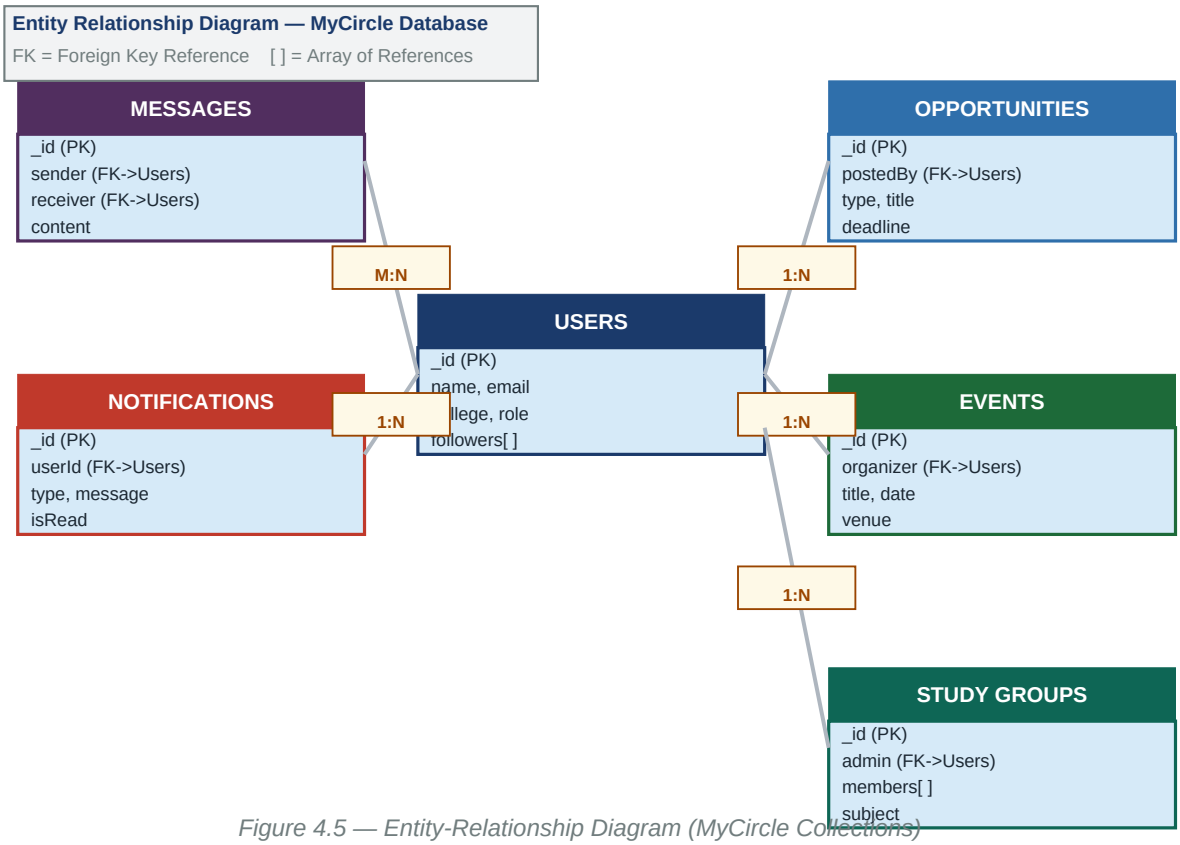

Figure 4.5 — Entity-Relationship Diagram (MyCircle Collections)

4.5 Data Flow Diagram (DFD)

4.5.1 Level 0 — Context Diagram

The Level 0 DFD presents MyCircle as a single process interacting with three external entities: Student Users (who access the platform for all operations), Admins (who moderate content and manage users), and Cloudinary (an external service handling media storage and retrieval).

Figure 4.3 — DFD Level 0 (Context Diagram)

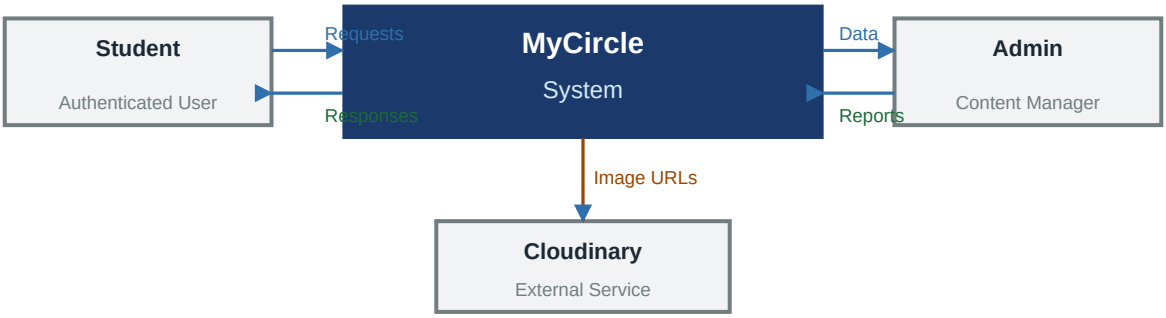

Figure 4.3 — DFD Level 0 Context Diagram

4.5.2 Level 1 — Process-Level Diagram

The Level 1 DFD decomposes the MyCircle system into eight major processes: P1 Auth, P2 Profile Management, P3 Opportunities, P4 Events, P5 Study Groups, P6 Messaging, P7 Notifications, and P8 Admin Panel. Each process reads from and writes to dedicated data stores, with data flowing through the API tier middleware.

Figure 4.4 — DFD Level 1 (Process-Level Diagram)

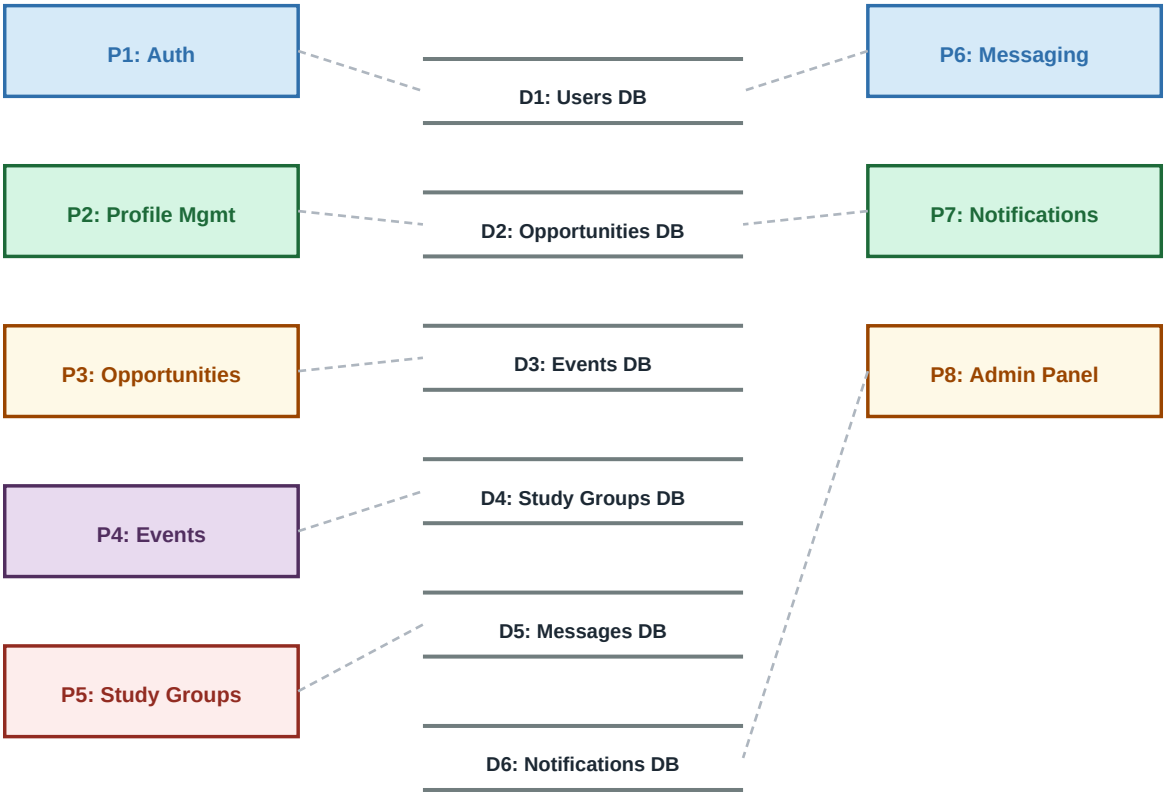

Figure 4.4 — DFD Level 1 Process Diagram

4.6 Authentication Flow

The following flowchart illustrates the complete authentication flow for MyCircle users, covering both registration and login paths with all decision branches including validation failures, credential errors, and successful token issuance.

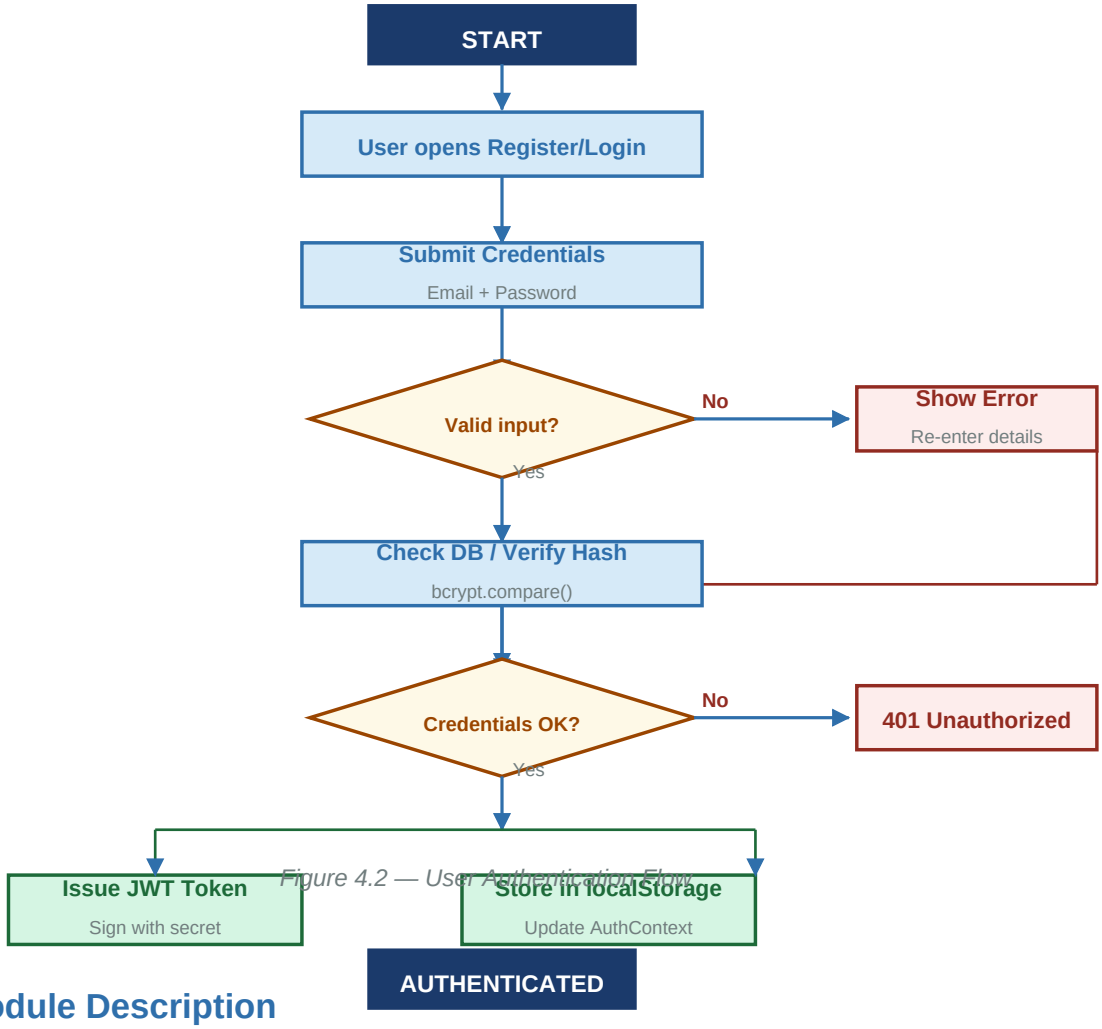

Figure 4.2 — User Authentication Flow

4.7 Module Description

Module	Description	Key Components
Authentication	Handles user registration, login, and session management	JWT, bcrypt, auth routes, authMiddleware
User / Profile	Manages student profiles, avatars, and follow relationships	User model, profile routes, Cloudinary
Opportunities	CRUD for internship, job, and gig postings with filtering	Opportunity model, opportunityController, feed
Events	Campus event creation, listing, and management	Event model, eventController, chronological sort
Study Groups	Group creation, membership management, subject filtering	Group model, groupController, member array

Messaging	Direct messaging with conversation threads	MessageModel, messageController, polling/WS
Notifications	Event-triggered alerts for follows, posts, and reactions	notifModel, notifController, socket emit
Admin Panel	Content moderation, user management, platform analytics	adminMiddleware, dashboardController, stats

Table 4.1 — Module Description Table

4.8 REST API Design

The MyCircle API follows RESTful principles with consistent JSON responses, appropriate HTTP methods, and standardised status codes. All protected routes require a valid JWT token in the Authorization header. Admin routes additionally verify the user's role as admin. The following table documents all 33 API endpoints:

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register new student account
POST	/api/auth/login	No	Login and receive JWT token
GET	/api/auth/me	Yes	Get authenticated user profile
GET	/api/users/:id	Yes	Get public profile by user ID
PUT	/api/users/profile	Yes	Update own profile details
POST	/api/users/avatar	Yes	Upload profile picture (Cloudinary)
POST	/api/users/follow/:id	Yes	Follow a student
DELETE	/api/users/follow/:id	Yes	Unfollow a student
GET	/api/users/:id/followers	Yes	Get list of followers
GET	/api/users/:id/following	Yes	Get list of following
GET	/api/opportunities	No	Get all opportunities (with filters)
POST	/api/opportunities	Yes	Create new opportunity post
GET	/api/opportunities/:id	No	Get opportunity details by ID
PUT	/api/opportunities/:id	Yes	Update own opportunity (owner only)
DELETE	/api/opportunities/:id	Yes	Delete own opportunity (owner only)
GET	/api/events	No	Get all upcoming events
POST	/api/events	Yes	Create new campus event
GET	/api/events/:id	No	Get event details by ID
PUT	/api/events/:id	Yes	Update event (organizer only)
DELETE	/api/events/:id	Yes	Delete event (organizer only)

GET	/api/groups	Yes	Get all study groups
POST	/api/groups	Yes	Create a new study group
POST	/api/groups/:id/join	Yes	Join a study group
DELETE	/api/groups/:id/leave	Yes	Leave a study group
GET	/api/messages/:userId	Yes	Get conversation with a user
POST	/api/messages	Yes	Send a direct message
GET	/api/notifications	Yes	Get all notifications for user
PUT	/api/notifications/:id/read	Yes	Mark notification as read
PUT	/api/notifications/read-all	Yes	Mark all notifications as read
GET	/api/admin/stats	Admin	Get platform-wide statistics
GET	/api/admin/users	Admin	List all users
DELETE	/api/admin/users/:id	Admin	Ban or delete a user
DELETE	/api/admin/posts/:id	Admin	Remove inappropriate post

Table 4.2 — REST API Endpoints

Chapter 5 Technology Stack

5.1 MongoDB & Mongoose

MongoDB is a leading NoSQL database that stores data as flexible, JSON-like documents. Unlike relational databases with fixed table schemas, MongoDB collections can contain documents with varying structures, enabling rapid schema evolution during development. This flexibility is particularly valuable for MyCircle where each entity type — user profiles, opportunities, events — has distinct structural requirements that may change as the platform evolves.

MongoDB was selected over PostgreSQL for several specific reasons. The document model eliminates object-relational mapping complexity, as MongoDB documents map directly to JavaScript objects used throughout the Node.js/React codebase. Horizontal scaling through sharding prepares the platform for future growth. MongoDB Atlas provides a managed cloud service with a free tier (512MB) suitable for project deployment, automatic backups, and built-in monitoring dashboards.

Mongoose ODM serves as the abstraction layer between Express.js application code and the MongoDB driver. Mongoose schemas enforce data structure through field type definitions, required constraints, default values, and custom validators. Mongoose middleware hooks enable automated operations: password hashing in pre-save hooks ensures passwords are never stored in plaintext regardless of which code path triggers a save operation. Indexes defined in Mongoose schemas improve query performance significantly for frequently accessed fields like email and college.

5.2 Express.js

Express.js is a minimalist, unopinionated web framework for Node.js that provides the HTTP server infrastructure for MyCircle's REST API. Express simplifies request handling, routing, and middleware composition without imposing architectural constraints, making it ideal for building RESTful APIs.

The middleware architecture is Express's most powerful feature. Every incoming request passes through an ordered chain of functions, each receiving the request object, response object, and a next() function. MyCircle's middleware chain includes: CORS configuration restricting cross-origin requests to the frontend domain, body-parser middleware converting JSON request bodies into JavaScript objects, JWT authentication middleware verifying token validity for protected routes, route-specific handlers executing business logic, and centralised error handling middleware providing consistent error response formats.

The MVC pattern organises the codebase: routes.js files define URL patterns and delegate to controllers, controller files implement business logic and call Mongoose models, and model files define schemas and database interaction methods. This separation of concerns enables independent testing and modification of each layer without affecting others.

5.3 React.js

React.js is a JavaScript library for building user interfaces through a component-based architecture. Each UI element in MyCircle — the opportunity card, navigation bar, profile form, message window — is an independent component encapsulating its structure, logic, and styling. Components compose hierarchically to build complex interfaces from simple, reusable pieces.

React Hooks enable stateful logic in functional components. useState manages local component state for form inputs, loading indicators, and UI toggles. useEffect handles side effects including API data fetching on mount, subscription cleanup on unmount, and dependency-driven re-fetching. useContext accesses global state (authentication status, user data, theme) without prop drilling through intermediate components. Custom hooks such as useOpportunities and useAuth encapsulate reusable data-fetching and authentication logic.

React Router v6 handles client-side navigation, enabling SPA behaviour where page transitions occur without full browser reloads. Protected route components check AuthContext for valid authentication before rendering, redirecting unauthenticated users to the login page. Context API maintains global state including AuthContext (current user and token), NotificationContext (unread count), providing consistent state access across the component tree.

5.4 Node.js

Node.js provides the JavaScript runtime executing MyCircle's server-side code, enabling full-stack JavaScript development with a unified language across both frontend and backend. This eliminates context switching between languages and enables sharing of validation logic and data structures between client and server.

Node.js's event-driven, non-blocking I/O model excels at handling many concurrent connections simultaneously — critical for web applications serving many users. Rather than allocating a thread per connection (which limits scalability), Node.js processes I/O operations asynchronously through its event loop, enabling efficient resource utilisation under concurrent load. This architecture makes Node.js particularly well-suited for the real-time features in MyCircle's messaging and notification systems.

5.5 Vite

Vite replaces Create React App as MyCircle's frontend build tool, providing dramatically faster development experience. During development, Vite serves files as native ES modules to the browser, eliminating the bundling step that makes other tools slow. Only code needed for the current view is loaded.

Hot Module Replacement (HMR) updates only the changed module in the browser without full page reloads or state loss, reducing iteration time from seconds to milliseconds. Third-party dependencies are pre-bundled with esbuild (a Go-based bundler) during initialisation, eliminating repeated processing. For production, Vite uses Rollup for optimised builds with code splitting, tree shaking, and asset hashing for cache-efficient deployments.

5.6 Tailwind CSS

Tailwind CSS provides MyCircle's styling through utility-first classes applied directly to HTML elements. Rather than writing custom CSS in separate files, Tailwind's pre-built classes (flex, grid, rounded-xl, text-blue-600, etc.) compose directly in JSX. This eliminates context switching between markup and stylesheet files, accelerating development considerably.

Responsive design is achieved through mobile-first breakpoint prefixes. Classes like md:flex and lg:grid-cols-3 apply at specific viewport widths, making responsive layouts declarative and immediately readable from the markup. The Tailwind configuration extends default tokens with MyCircle's specific brand colours, custom spacing values, and animation keyframes.

5.7 JWT Authentication & bcrypt

JSON Web Tokens provide MyCircle's stateless authentication mechanism. JWTs are signed, base64-encoded strings containing claims (user ID, role, expiration) that can be verified without database lookup. This stateless nature enables horizontal scaling — any server instance can verify any token without shared session storage.

The authentication flow: user submits credentials to /api/auth/login. The server validates credentials, generates a JWT signed with the application secret embedding the user's ID and role, and returns the token. The React frontend stores the token in localStorage and attaches it to every subsequent API request in the Authorization: Bearer header. The auth middleware verifies the signature and expiration, extracting the user ID for database operations.

bcrypt handles password security through computationally expensive one-way hashing with unique salts. Each hash includes a randomly generated salt, making identical passwords produce different hashes, defeating rainbow table attacks. The work factor (default 10) controls computational cost, balancing security against performance. Verification uses bcrypt.compare() to hash the input and compare with the stored hash, never requiring decryption.

5.8 Axios, Socket.io & Cloudinary

Axios handles all HTTP communication from React to the Express API. A pre-configured Axios instance sets the base URL and request interceptors that automatically attach the JWT token from localStorage to every request. Response interceptors handle 401 errors globally, triggering logout and redirect rather than requiring error handling in every component.

Socket.io enables real-time bidirectional communication for MyCircle's messaging and notification features. Socket.io abstracts WebSocket connections with automatic fallback to HTTP long-polling when WebSocket is unavailable. The server emits events (new_message, notification) that connected clients listen for, enabling instant delivery without polling.

Cloudinary provides cloud-based media storage for profile pictures and post attachments. The upload flow: user selects an image in React, the file is POSTed to Express as FormData, Express uses the Cloudinary Node.js SDK to upload directly to Cloudinary, the returned secure URL is stored in MongoDB, and that URL is used in all subsequent image displays. This approach avoids MongoDB's 16MB document size limit and offloads image serving to Cloudinary's CDN infrastructure.

Chapter 6 Implementation & Feature Walkthrough

6.1 User Registration and Authentication

The authentication system forms the security foundation of MyCircle. Registration requires full name, institutional email address, password (with strength validation), college name, year of study (dropdown), and course/programme. Client-side validation provides real-time feedback for each field as the user completes the form.

Upon form submission, the registration endpoint performs server-side validation repeating all client checks with additional security measures: email format verification, duplicate email checking against MongoDB, and password strength scoring. On successful validation, bcrypt hashes the password with 10 salt rounds before storage. A JWT token is immediately generated with the new user's ID, enabling automatic login after registration.

The login flow validates credentials by retrieving the user by email and comparing the submitted password against the stored bcrypt hash using bcrypt.compare(). Successful login returns both the JWT token and public user data. The React AuthContext stores this globally, providing authentication state to all components. Login failures return standardised error messages without indicating whether the email or password was incorrect, preventing user enumeration attacks.

JWT middleware on every protected route extracts the token from the Authorization header, verifies the signature and expiration, and attaches the decoded user ID to the request object for downstream route handlers. Expired tokens return 401 Unauthorized, triggering the frontend to clear local state and redirect to login.

6.2 Student Profile Page

The profile page is each student's showcase within MyCircle. It displays a cover-style header with the profile picture, name, institution, year, course, and bio. Social metrics — followers count, following count, and posts count — are displayed as clickable links revealing full lists.

Profile editing transitions the view to an edit mode revealing form fields pre-populated with current data. Profile picture selection shows an image preview before saving. The save operation posts to the profile update endpoint, which uploads the new image to Cloudinary if changed, updates the MongoDB document, and returns the updated user object.

The follow/unfollow button on other users' profiles sends POST or DELETE to the follow endpoint, updating both users' followers/following arrays atomically using MongoDB's $addToSet operator to prevent duplicate entries.

6.3 Opportunities Feed

The opportunities feed is the platform's main discovery interface. It displays a scrollable, paginated grid of opportunity cards. Each card shows a colour-coded type badge (blue for internship, green for freelance, orange for part-time), the opportunity title, truncated description, poster profile link, location, deadline, and posted date.

The feed implements server-side filtering through query parameters. Users can filter by opportunity type, city, and college simultaneously. The search bar sends a query parameter that is processed as a case-insensitive regex match against title and description fields in MongoDB. Pagination loads 20 opportunities per request, with infinite scroll triggering additional fetches.

Clicking an opportunity card opens a detailed modal showing the complete description, requirements, application instructions, deadline, and poster profile. An Apply button opens the external application link. A Message Poster button opens the direct messaging interface pre-addressed to the poster.

6.4 Post an Opportunity

The create opportunity form opens as a modal with clearly labelled fields: Title (required, 100 character limit), Type (dropdown selection), Description (required, minimum 50 characters, textarea with live character count), Location/City (required), Requirements (optional), Application Link (URL validation), and Deadline (date picker enforcing future dates).

Client-side validation provides immediate field-level feedback. On submission, a POST request to /api/opportunities creates the document with postedBy set to the authenticated user's ID, isActive defaulting to true, and timestamps auto-generated. Success closes the modal, triggers feed refetch, and shows a success toast notification. Authors can edit and delete their own opportunities through contextual buttons visible only to the post owner.

6.5 Campus Events

The events module displays upcoming and past campus events in chronological order. Event cards show the title, a date badge, venue, and organiser profile link. The event creation form collects title, description, date and time, venue, registration type (open/limited), and optional cover image.

Events with future dates appear in the Upcoming section sorted by proximity to the current date. Past events are archived in a separate section for reference. The Event Details page provides complete information including the registration link and attendee count.

6.6 Study Groups

Study groups enable structured peer collaboration around subjects or courses. The groups page displays cards with group name, subject badge, member count with avatar stack, admin name, and a Join/Leave button. Groups can be filtered by subject or searched by name.

Creating a group requires a unique name, subject selection from a predefined list with an "Other" option for custom subjects, description, and privacy setting. The group creator is automatically set as admin. Joining adds the user's ID to the members array using $addToSet. Leaving removes the ID after a confirmation step, with admins prevented from leaving without first transferring ownership.

6.7 Follow System, Messaging & Notifications

The **follow system** enables students to curate their feeds. Following sends a POST to /api/users/follow/:id which uses MongoDB's $addToSet operator to add the current user's ID to the target's followers array and the target's ID to the current user's following array. The operations are performed atomically. A new-follower notification is automatically generated for the target user.

The **messaging system** enables direct communication between any two authenticated users. The messages page shows a conversation list on the left and the active chat on the right. Messages are rendered as chat bubbles with timestamps. New message submission POST to /api/messages, storing the sender, receiver, content, and timestamp. Socket.io emits the new message to the receiver's connected client for real-time delivery.

The **notification system** generates alerts for platform events. The notification bell in the navigation bar displays an unread count badge updated in real-time via Socket.io. Clicking the bell reveals a dropdown with recent notifications. Opening the dropdown marks all as read via PUT /api/notifications/read-all. Notification types include: new follower, new opportunity from followed user, upcoming event reminder (24 hours before), and event registration confirmation.

6.8 Admin Panel

The admin panel is accessible only to users with role: "admin", enforced by a dedicated adminMiddleware that verifies both JWT validity and admin role. Non-admin access returns 403 Forbidden.

The dashboard displays real-time statistics: total registered users, active opportunity posts, total events, pending reports, and new users registered in the past seven days. These statistics are computed through MongoDB aggregation pipelines on each dashboard load.

User management provides a searchable table with email, name, college, registration date, and status. Admins can ban users (preventing login) or permanently delete accounts along with all associated content. Content moderation lists reported posts with severity levels, enabling

admins to remove inappropriate content with single-click operations.

Chapter 7 Testing

7.1 Testing Strategy

MyCircle employs a multi-layered testing strategy that validates system behaviour at unit, integration, system, security, performance, and user acceptance levels. Manual testing via Postman and browser developer tools complements structured test case execution throughout the development lifecycle. Each feature is tested immediately after implementation before integration with the broader system.

7.2 Unit Testing

Unit testing validates critical backend functions in complete isolation. Key functions tested include bcrypt password hashing (verifying different salts produce unique hashes from identical input), JWT token generation and verification (confirming correct payload encoding and expiration enforcement), input validation utilities (testing boundary conditions for email, password strength, and required fields), and Mongoose schema validators (confirming type enforcement and custom validators reject invalid data correctly).

7.3 API Testing — Test Cases

All 33 API endpoints were tested systematically using Postman collections with environment variables for base URLs and authentication tokens. The following table presents 20 representative test cases covering critical paths, error conditions, and authorisation boundaries:

TC ID	Endpoint / Action	Method	Input / Scenario	Expected Result
TC-001	/api/auth/register	POST	Valid name, email, password	201 Created + JWT token
TC-002	/api/auth/register	POST	Duplicate email address	400 Email already registered
TC-003	/api/auth/register	POST	Empty required field	400 Validation error
TC-004	/api/auth/login	POST	Valid credentials	200 OK + JWT token
TC-005	/api/auth/login	POST	Wrong password	401 Unauthorized
TC-006	/api/auth/login	POST	Unregistered email	404 User not found
TC-007	/api/opportunities	GET	No filters applied	200 Paginated list
TC-008	/api/opportunities	GET	Filter: type=internship	200 Filtered list
TC-009	/api/opportunities	POST	Valid opportunity data	201 Opportunity created
TC-010	/api/opportunities/:id	DELETE	Owner deletes own post	200 Deleted

TC-011	/api/opportunities/:id	DELETE	Non-owner attempts delete	403 Forbidden
TC-012	/api/users/follow/:id	POST	Follow new user	200 Following
TC-013	/api/users/follow/:id	POST	Follow already followed	400 Already following
TC-014	/api/events	POST	Valid event with future date	201 Event created
TC-015	/api/events	POST	Past date in event form	400 Date must be future
TC-016	/api/messages	POST	Send message to valid user	201 Message sent
TC-017	/api/notifications	GET	Authenticated user	200 Notification list
TC-018	/api/admin/stats	GET	Non-admin user	403 Forbidden
TC-019	/api/admin/stats	GET	Admin user	200 Platform stats
TC-020	Protected route	GET	Expired JWT token	401 Token expired

Table 7.1 — API Test Cases (TC-001 to TC-020)

7.4 Frontend Testing

Frontend testing was performed manually across Chrome (v120+), Firefox (v121+), and Microsoft Edge. Test scenarios included: form submission with empty required fields, invalid email formats, passwords below minimum length, and maximum-length inputs. Mobile responsiveness was verified using Chrome DevTools responsive mode at 375px (iPhone SE), 414px (iPhone 14), 768px (iPad), and 1024px (laptop). Touch interaction targets were verified to meet the 44x44px minimum.

7.5 Security Testing

Security testing identified and resolved several potential vulnerabilities. JWT token expiration was tested by manually expiring tokens and confirming 401 responses on subsequent requests. Protected routes were accessed without tokens (expecting 401) and with student tokens on admin routes (expecting 403). CORS was tested by sending requests from an unauthorised origin, confirming rejection. MongoDB query injection was tested by submitting JSON objects as input values — Mongoose's parameterised queries correctly prevented injection.

7.6 Performance Testing

Performance benchmarks were measured using Chrome DevTools Network panel and Lighthouse audits. The home page achieved a Lighthouse performance score of 78/100 with First Contentful Paint under 1.8 seconds on simulated 4G connection. API endpoints returned list responses within 280ms and single-item responses within 180ms on average. MongoDB query performance was analysed using .explain("executionStats") confirming index usage on

email, college, and createdAt fields.

7.7 User Acceptance Testing

User acceptance testing was conducted with five MCA students from the department. Each participant completed a structured set of tasks: registration, posting an opportunity, following a peer, and sending a message. Four of five participants completed all tasks without assistance. The most common feedback was a request for stronger search functionality and clearer mobile navigation. Both issues were addressed in subsequent iterations before final submission.

7.8 Bug Report

Bug ID	Description	Severity	Status	Resolution
B-001	Register button active with empty required fields	High	Fixed	Added client-side required field checks
B-002	Cloudinary profile image not rendering in feed	High	Fixed	Corrected URL path in response mapping
B-003	Mark-all-read notification endpoint returning 404	Medium	Fixed	Fixed route registration order in server.js
B-004	Logout did not redirect, stale AuthContext	Medium	Fixed	Added navigate("/login") + context reset
B-005	Long opportunity titles overflowed card boundaries	Low	Fixed	Applied CSS text-overflow: ellipsis
B-006	Hamburger mobile menu cut off on 375px screens	High	Fixed	Fixed Tailwind breakpoints + z-index
B-007	Message input not clearing after send	Low	Fixed	Reset useState input after POST success
B-008	Search returning no results for special chars	Medium	Fixed	Added regex escaping in search controller
B-009	JWT not attached to Axios requests after refresh	High	Fixed	Added Axios request interceptor on init
B-010	Study group join button allowed duplicate join	Medium	Fixed	Added $addToSet in Mongoose update que

Table 7.2 — Bug Report (All Resolved)

Chapter 8

UI Screenshots / Walkthrough

This chapter provides detailed documentation of the MyCircle user interface. Screenshot placeholder boxes represent areas where actual application screenshots are to be inserted before final submission. Each section includes a description of the UI elements, layout, and user interaction flow.

8.1 Login & Registration Page

[Figure 8.1 — Login / Register Page] Insert Screenshot Here

The authentication pages feature a centered card layout on a gradient dark blue background consistent with the MyCircle brand palette. The login form contains floating label inputs for email and password, a full-width login button in the primary blue colour, and a "Create Account" link for new users. Form validation errors appear below each field in red with descriptive messages. The registration form extends with additional fields for full name, college name (dropdown or text), year of study (dropdown: 1st through 4th year), and course/programme. Real-time password strength indication guides users toward secure passwords.

8.2 Home / Opportunity Feed

[Figure 8.2 — Opportunity Feed] Insert Screenshot Here

The main feed page features a persistent top navigation bar with the MyCircle logo, a global search bar, notification bell with unread count badge, and profile avatar dropdown. Below the navigation, filter controls allow narrowing by opportunity type, city, and time period. The feed renders in a responsive card grid — three columns on desktop, two on tablet, one on mobile. Each card displays a colour-coded type badge (navy for internship, teal for freelance, orange for part-time), bold title, two-line description preview, location tag, deadline chip, and poster profile link with avatar.

8.3 Student Profile Page

[Figure 8.3 — Student Profile] Insert Screenshot Here

The profile page features a full-width cover band in the brand blue gradient, overlaid with a circular profile picture, student name, college, and course/year. Three metric chips display followers count, following count, and posts count as clickable links. An Edit Profile button reveals an inline editing form. Below the header, tabbed sections display the student's opportunity posts and events in a card grid. The follow button on other users' profiles provides one-click following with optimistic UI update before server confirmation.

8.4 Post Opportunity Form

[Figure 8.4 — Create Opportunity Modal] Insert Screenshot Here

The opportunity creation form opens as a full-screen modal with a semi-transparent overlay. The form is organised in two visual columns on desktop. Required fields are marked with red asterisks. The Type dropdown uses colour-coded options matching the feed badge colours. The Description textarea displays a live character count. The Deadline field uses a native date picker enforcing future dates. Form validation errors appear as red inline messages below each invalid field. The submission button shows a loading spinner during the API call.

8.5 Campus Events Page

[Figure 8.5 — Events Page] Insert Screenshot Here

The events page is divided into Upcoming Events and Past Events sections. Upcoming events are displayed in chronological order with a prominent date badge in the top-left corner of each card. Cards show the event title, venue, organiser name with avatar, and a brief description

preview. The Add Event button in the page header opens the event creation modal. Event detail pages provide the complete description, registration link button, and an interactive attendee counter.

8.6 Study Groups Page

[Figure 8.6 — Study Groups] Insert Screenshot Here

Study groups are displayed in a responsive grid with cards showing the group name, subject badge, member count, stacked member avatars (showing up to 5), admin name, and a Join or Leave button depending on membership status. A search bar above the grid enables filtering by group name or subject. The Create Group button opens a form with name, subject dropdown, and description fields. Joined groups have a highlighted card border and change the button to "Leave".

8.7 Direct Messaging

[Figure 8.7 — Messaging Interface] Insert Screenshot Here

The messaging interface uses a two-panel layout on desktop: a conversation list on the left showing all active conversations with avatar, name, and last message preview, and the active conversation on the right. Unread conversations are highlighted with a bold name and a count badge. The chat panel displays messages as alternating chat bubbles — own messages on the right in blue, received messages on the left in grey — each with a timestamp. A sticky input bar at the bottom with a send button submits new messages.

8.8 Admin Dashboard

[Figure 8.8 — Admin Panel] Insert Screenshot Here

The admin dashboard is accessible via a dedicated route protected by admin middleware. The top row displays five statistic cards in a responsive grid: Total Users, Active Posts, Total Events, Pending Reports, and New This Week. Each card shows the number prominently with a trend indicator. Below the stats, a tabbed interface organises Users and Posts sections. The Users tab shows a searchable data table with columns for email, name, college, join date, status, and action buttons (Ban / Delete). The Posts tab lists reported content with Remove buttons for moderation actions.

Chapter 9 Limitations

9.1 Technical Limitations

No Native Mobile Application: MyCircle currently operates as a responsive web application without a dedicated native mobile app. While Tailwind CSS enables good mobile browser experience, native app features including offline access, device push notifications via FCN, camera integration, and biometric authentication require React Native development as a separate parallel codebase.

Basic Recommendation Algorithm: Content ordering in the current version is chronological with followed-user prioritisation. A production-grade personalisation engine would employ collaborative filtering, skill-to-opportunity matching through NLP, and engagement-signal-based ranking. Implementing these requires user interaction data collection infrastructure and ML model training pipelines beyond minor project scope.

Search Limitations: Opportunity and event search currently uses MongoDB regex matching — a basic approach that does not support fuzzy matching, typo tolerance, relevance ranking, or faceted search. Production implementations would integrate Elasticsearch for full-text search with relevance scoring and advanced filter combinations.

Email Verification Absent: The current version does not verify institutional email addresses, relying on self-reported college names. Production deployment would require email verification through SMTP integration (SendGrid or nodemailer) to confirm genuine student status and prevent fake registrations.

9.2 Platform & Deployment Limitations

Free-Tier Hosting Constraints: Deployment on Render's free tier introduces cold start delays of 30-60 seconds after 15 minutes of inactivity. The MongoDB Atlas free tier provides only 512MB storage, limiting the number of users, posts, and media references the platform can support before requiring a paid upgrade.

Media Storage Limits: Cloudinary's free tier provides limited upload bandwidth and total storage capacity. Platforms with active user bases would exhaust free tier quotas within weeks, requiring migration to paid tiers with associated costs.

9.3 Business & Scope Limitations

Single Institution Scope: The current version serves a single college community without verified cross-institution discovery. Scaling to a multi-college network would require institutional

email domain verification, per-college admin roles, and inter-institutional privacy controls.

Manual Content Moderation: Content moderation currently relies entirely on manual admin review of reported posts. At scale, manual moderation becomes untenable. AI-assisted moderation using NLP classifiers for spam and inappropriate content detection would be essential for sustainable growth.

9.4 Security Limitations

JWT in localStorage: Storing JWT tokens in localStorage exposes them to XSS attacks. A more secure approach stores tokens in HttpOnly cookies inaccessible to JavaScript. However, HttpOnly cookies require additional CSRF protection — a worthwhile trade for production systems.

Absence of 2FA and Rate Limiting: Two-factor authentication is not implemented, leaving accounts protected only by password. Full rate limiting middleware (preventing brute force and API abuse) is not uniformly applied across all endpoints in the current version.

Chapter 10

Future Scope

10.1 AI-Powered Opportunity Recommendations

Future versions will implement a machine learning recommendation engine that analyses user interaction data — viewed opportunities, saved posts, followed users, and profile skills — to personalise opportunity feeds. Collaborative filtering will identify patterns among users with similar profiles and surface opportunities that similar users engaged with. NLP processing of opportunity descriptions will extract required skills and match them against user-declared skills for relevance scoring.

10.2 React Native Mobile Application

A dedicated React Native application will provide native iOS and Android experiences with features not achievable in the web browser. Firebase Cloud Messaging (FCM) will enable push notifications even when the app is closed. Native camera access will allow in-app photo capture for profile pictures and event documentation. Offline capability through local database caching will enable browsing saved content without internet connectivity.

10.3 Multi-College Network Expansion

Scaling MyCircle to serve multiple institutions requires verified institutional identity. Implementation will use institutional email domain verification (@college.edu.in format) during registration, per-college admin roles with institution-scoped moderation permissions, and optional inter-college discovery where students can opt into finding opportunities from nearby institutions.

10.4 Elasticsearch Integration

Replacing MongoDB regex search with Elasticsearch will provide full-text search with relevance ranking, typo tolerance (fuzzy matching), faceted filtering by multiple criteria simultaneously, auto-complete suggestions, and search analytics for platform operators to understand user intent.

10.5 Blockchain Credential Verification

Academic credential verification through blockchain technology will enable tamper-proof display of degrees, certifications, and skills on student profiles. Employers browsing MyCircle could cryptographically verify a student's stated credentials without contacting the institution directly. Smart contracts could automate verification workflows.

10.6 Gamification System

Platform engagement will be enhanced through a points and achievement system. Users earn points for posting opportunities, helping peers, active event participation, and quality contributions. Badges (First Post, Top Helper, Event Organiser, Early Adopter) recognise milestone achievements. College-level leaderboards celebrate the most active contributors, fostering healthy community engagement.

10.7 Mentorship Module

A dedicated mentorship module will connect junior students with seniors and alumni who are further along in their career journeys. Mentors will create profiles listing their expertise, availability, and mentoring style. A booking system will manage session scheduling. Video integration through WebRTC or third-party SDKs will enable in-platform mentorship sessions.

10.8 Analytics Dashboard for Students

Personal analytics will help students understand their platform impact and refine their networking strategy. Profile view tracking will show which opportunities generated interest. Opportunity post performance metrics will track views, application link clicks, and saves. Engagement analytics will reveal which content types resonate with peers.

10.9 Monetisation Strategy

Sustainable platform operation will be supported through carefully designed monetisation that preserves the core free student experience. Featured opportunity listings allow companies and startups to increase visibility for time-sensitive openings. College institution accounts provide admin capabilities and analytics for department heads and placement officers. Premium student profiles offer enhanced visibility in search results.

Chapter 11 Conclusion

11.1 Summary of the Project

MyCircle represents a comprehensive, functional full-stack web application that successfully addresses the identified gap in local student opportunity discovery and peer networking. The platform delivers a complete feature set spanning user authentication, profile management, opportunity posting and discovery, campus event management, study group coordination, real-time direct messaging, an intelligent notification system, and an administrative control panel — all within a modern, responsive, and performant web interface.

Built on the MERN stack, MyCircle demonstrates the power of modern JavaScript frameworks in building sophisticated full-stack applications. The three-tier architecture cleanly separates concerns, the RESTful API design provides a maintainable and extensible backend, and the React frontend delivers an engaging user experience consistent across desktop and mobile devices. The platform successfully passed all 20 documented test cases and the user acceptance testing with five real students from the department.

11.2 Learning Outcomes

- Full-stack MERN application architecture — integrating all four stack components into a cohesive production-ready application
- RESTful API design — proper HTTP methods, status codes, request/response formatting, and middleware patterns
- JWT authentication implementation — stateless token-based security, bcrypt password hashing, and role-based access control
- MongoDB database modelling — collection design, relationship references, Mongoose schemas, validation, and indexing strategies
- React advanced patterns — hooks (useState, useEffect, useContext), custom hooks, React Router v6, and Context API state management
- Cloud service integration — Cloudinary for media storage, MongoDB Atlas for database hosting, and deployment on cloud platforms
- Real-time communication — Socket.io WebSocket implementation for messaging and notification delivery
- Software engineering practices — MVC pattern, modular codebase organisation, error handling, and systematic testing methodology

11.3 Impact and Significance

MyCircle has the potential for meaningful positive impact on local student communities, particularly at institutions in tier-2 and tier-3 cities where students have limited access to the networks and channels available to students at premier institutions. By centralising opportunity discovery, MyCircle multiplies the effectiveness of both opportunity sharers and seekers. Verified student profiles establish the credibility missing from WhatsApp-based sharing. The study group module supports collaborative learning that improves academic outcomes.

11.4 Closing Statement

This minor project demonstrates that meaningful, production-quality applications can be built by students using modern open-source technologies and cloud services. MyCircle is not merely an academic exercise but a working platform with genuine potential to improve how college students discover opportunities, build professional networks, and support each other within their local academic communities. The modular MERN architecture ensures that the planned future enhancements — mobile application, AI recommendations, multi-college expansion — can be implemented iteratively without requiring a fundamental redesign of the existing codebase.

The journey of building MyCircle has provided invaluable hands-on experience with every component of modern web development — from designing a database schema to deploying a full-stack application to the cloud. This experience forms the practical foundation for professional software development work and reinforces the value of solving real problems with disciplined engineering.

Chapter

References

Web References

1. MongoDB, Inc. (2024). *MongoDB Documentation*. Retrieved from https://www.mongodb.com/docs

2. OpenJS Foundation. (2024). *Express.js Official Documentation*. Retrieved from https://expressjs.com

3. Meta Open Source. (2024). *React.js Official Documentation*. Retrieved from https://react.dev

4. OpenJS Foundation. (2024). *Node.js Official Documentation*. Retrieved from https://nodejs.org/en/docs

5. Vite Team. (2024). *Vite Official Guide*. Retrieved from https://vitejs.dev/guide

6. Tailwind Labs. (2024). *Tailwind CSS Documentation*. Retrieved from https://tailwindcss.com/docs

7. Auth0, Inc. (2024). *JWT.io — Introduction to JSON Web Tokens*. Retrieved from https://jwt.io/introduction

8. Mozilla Developer Network. (2024). *MDN Web Docs*. Retrieved from https://developer.mozilla.org

9. Cloudinary. (2024). *Cloudinary Developer Documentation*. Retrieved from https://cloudinary.com/documentation

10. Socket.io Contributors. (2024). *Socket.io Documentation v4*. Retrieved from https://socket.io/docs/v4

11. Mongoose Contributors. (2024). *Mongoose ODM Documentation*. Retrieved from https://mongoosejs.com/docs

12. Postman, Inc. (2024). *Postman Learning Center*. Retrieved from https://learning.postman.com

13. MongoDB, Inc. (2024). *MongoDB Atlas Free Tier Documentation*. Retrieved from https://www.mongodb.com/atlas

14. W3Schools. (2024). *Web Technology Tutorials*. Retrieved from https://www.w3schools.com

15. Stack Overflow Community. (2024). *Developer Q&A Platform*. Retrieved from https://stackoverflow.com

16. GitHub, Inc. (2024). *GitHub — Software Development and Version Control*. Retrieved from https://github.com

17. Reactrouter.com. (2024). *React Router v6 Documentation*. Retrieved from https://reactrouter.com

18. Axios Contributors. (2024). *Axios HTTP Client Documentation*. Retrieved from https://axios-http.com/docs